

Manual Técnico:



Índice:

1. JUSTIFICACIÓN Y PROPUESTA DEL PROYECTO	2
2. ARQUITECTURA DE SOFTWARE (DESIGN SYSTEM)	2
2.1. Capa de Componentes Atómicos	2
2.2. Lógica Desacoplada (Custom Hooks)	3
3. INGENIERÍA DE DATOS (SUPABASE & POSTGRES)	3
3.1. Esquema Relacional y RBAC	3
3.2. Seguridad en Capa de Datos (RLS)	3
4. REQUISITOS DEL SISTEMA	4
4.1. Requisitos Funcionales (RF)	4
4.2. Requisitos No Funcionales (RNF)	5
5. CONCLUSIÓN TÉCNICA	6

1. JUSTIFICACIÓN Y PROPUESTA DEL PROYECTO

Kore Manager surge como una solución tecnológica a la ineficiencia detectada en la gestión de instalaciones deportivas públicas. La propuesta se basa en tres pilares fundamentales:

1. **Centralización:** Un solo punto de control para inventario, usuarios y reservas.
 2. **Accesibilidad:** Interfaz diseñada para que cualquier ciudadano, independientemente de su capacidad técnica, pueda interactuar con la administración.
 3. **Transparencia:** Control de stock y uso de instalaciones auditable en tiempo real.
-

2. ARQUITECTURA DE SOFTWARE (DESIGN SYSTEM)

A diferencia de otros proyectos que utilizan plantillas prefabricadas, Kore Manager implementa un **Sistema de Diseño propio (DRY)**.

2.1. Capa de Componentes Atómicos

Se han desarrollado componentes base en React para garantizar la coherencia visual y técnica:

- **Input.jsx:** Encapsula la validación de estados y el manejo de iconos (Lucide).
- **Button.jsx:** Gestiona variantes (Primary, Secondary, Ghost) y estados de carga (*loading states*) para mejorar el feedback visual al usuario.
- **Card.jsx:** Contenedor semántico que unifica la estética de toda la plataforma.

2.2. Lógica Desacoplada (Custom Hooks)

Siguiendo los principios de **Clean Architecture**, la lógica de conexión con la base de datos no reside en la interfaz. Se utilizan *Custom Hooks* como `useProfile.js` para:

- Gestionar el estado global del usuario.
 - Interceptar errores de conexión y transformarlos en notificaciones amigables (Toasts).
 - Optimizar el rendimiento evitando re-renders innecesarios.
-

3. INGENIERÍA DE DATOS (SUPABASE & POSTGRES)

El núcleo del proyecto reside en una base de datos PostgreSQL alojada en Supabase, aprovechando su motor de políticas de seguridad.

3.1. Esquema Relacional y RBAC

El sistema implementa un **Control de Acceso Basado en Roles (RBAC)** mediante la tabla roles. Esto permite una jerarquía clara:

- **CIUDADANO:** Acceso a perfil y reservas propias.
- **CONSERJE:** Permisos de escritura en la tabla inventario para control de material.
- **ADMIN:** Acceso total al panel de gestión y auditoría de incidencias.

3.2. Seguridad en Capa de Datos (RLS)

Para cumplir con la **RGPD**, se han definido políticas de **Row Level Security**:

-- Ejemplo de Política RLS implementada:

```
CREATE POLICY "Usuarios solo ven su propio perfil"
```

```
ON public.profiles FOR SELECT
```

```
USING (auth.uid() = id);
```

Esto garantiza que ni siquiera un ataque por consola en el frontend pueda extraer datos de otros usuarios.

4. REQUISITOS DEL SISTEMA

4.1. Requisitos Funcionales (RF)

- **RF-01: Gestión de Autenticación:** Registro e inicio de sesión seguro mediante Supabase Auth, permitiendo la recuperación de contraseñas y validación de sesiones.
- **RF-02: Gestión de Perfiles:** Cada usuario puede visualizar y editar su información personal (nombre, teléfono), vinculada de forma única a su identidad de autenticación.
- **RF-03: Control de Inventario:** Visualización dinámica del material deportivo. Permite a los conserjes aumentar o disminuir el stock y categorizar los productos (balones, redes, etc.).
- **RF-04: Sistema de Reservas:** Módulo para la selección de instalaciones (pistas de fútbol, pádel, tenis) con validación de disponibilidad horaria.
- **RF-05: Gestión de Incidencias:** Formulario para que ciudadanos y personal reporten desperfectos, permitiendo adjuntar descripción y estado de la avería.
- **RF-06: Panel de Administración:** Interfaz exclusiva para el administrador para gestionar usuarios, asignar roles y supervisar el estado global del complejo.

- **RF-07: Notificaciones en tiempo real:** Feedback inmediato al usuario mediante avisos (Toasts) tras realizar acciones como guardar perfil o actualizar stock.

4.2. Requisitos No Funcionales (RNF)

- **RNF-01: Seguridad (Crítico):**
 - Implementación de cifrado SSL en todas las comunicaciones.
 - Uso de **Row Level Security (RLS)** para asegurar que los datos sensibles solo sean accesibles por sus propietarios o administradores.
- **RNF-02: Rendimiento:** El sistema debe responder a las consultas de base de datos en menos de 500ms para garantizar una experiencia fluida. Optimización de imágenes y carga de componentes mediante *Lazy Loading*.
- **RNF-03: Escalabilidad:** Arquitectura basada en microservicios (a través de Supabase) que permite el crecimiento de la base de datos sin pérdida de rendimiento.
- **RNF-04: Disponibilidad:** Garantizada mediante el despliegue en infraestructuras Cloud, minimizando los tiempos de inactividad por mantenimiento.
- **RNF-05: Usabilidad (UX/UI):** Interfaz intuitiva con diseño "Dark Mode" de alto contraste, siguiendo principios de accesibilidad y diseño adaptativo (Responsive Design) para dispositivos móviles.
- **RNF-06: Portabilidad:** Aplicación ejecutable en cualquier navegador moderno (Chrome, Firefox, Safari, Edge) sin necesidad de instalaciones adicionales por parte del usuario.

5. CONCLUSIÓN TÉCNICA

El proyecto no es solo una web de reservas; es una infraestructura escalable que utiliza las últimas tecnologías del mercado (Stack PERN: Postgres, Express, React, Node) simplificadas a través de Supabase para ofrecer un producto final de nivel empresarial, seguro y fácil de mantener.

